김선우

Backend / Full-stack Developer

장바구니 중복 반영 방지, 인증 갱신과 로그아웃 충돌, 배포 실패 복구를 Spring Boot·Next.js·배포 스크립트와 테스트로 검증한 풀스택 웹 서비스입니다.

첫 장에서 확인할 대표 구현

- **장바구니 중복 반영 방지** — DB UNIQUE와 동일 트랜잭션 기록
- **인증 갱신과 로그아웃 충돌** — Refresh 세대 폐기와 늦은 응답 무시
- **배포 실패 복구** — 준비 검사 실패 시 이전 서비스 복구·컨테이너 보존

서비스 https://swportfolioservices.com · 저장소 KimSunWooo/portfolio

근거 README: 핵심 문제 해결 · 장애 기록: 2026-10-13 backend readiness

kssgss@gmail.com · 010-7382-4100

재현 증상 → 원인 → 핵심 코드 → 검증 결과

1. 장바구니 중복 반영 방지

재현: 로그인 병합 응답을 못 받아 같은 요청을 다시 보내면 수량이 중복 합산될 수 있었습니다.

원인: 프로세스 메모리만으로 완료 여부를 구분하면 재시작·다중 인스턴스에서 같은 요청을 다시 적용합니다.

핵심 코드: `CartService`, `cart_sync_requests` UNIQUE, 본문 SHA-256.

검증: `CartSyncMySqlTest`, Playwright `merge-retry.spec.ts`. main에 커밋·테스트로 반영되어 있습니다.

2. 인증 갱신과 로그아웃 충돌

재현: 로그아웃 전에 시작한 Refresh가 늦게 도착하면 로그인 상태가 되살아날 수 있었습니다.

원인: Access는 메모리, Refresh는 쿠키·Redis이며 세대 번호 없이 늦게 온 응답을 그대로 적용했습니다.

핵심 코드: `frontend/src/lib/api.ts`의 `authGeneration`, `RefreshSessionService`, `users.session_epoch`.

검증: `SessionLifecycleMySqlTest`, `authSession.test.mts`, Playwright `late-refresh.spec.ts`. Access Token은 만료(30분)까지 유효합니다.

3. 배포 실패 복구

재현: 2026-10-13 배포에서 새 백엔드 준비 검사가 실패하고 `rollback from phase backend`가 남았습니다.

원인: 운영은 `ddl-auto=validate`인데 새 엔티티 테이블을 Flyway가 만들지 않아 기동이 실패했습니다.

핵심 코드: `deploy.sh` 준비 검사·슬롯 전환·실패 컨테이너 보존, Flyway baseline 0.

검증: `ops/tests/run-deploy-checks.sh`, 장애 기록의 가설·수정, Actions mock 검사. 복구는 마이그레이션을 되돌리지 않으며 프런트 교체 중 짧은 중단이 있습니다.

> 캐시 측정: 격리 환경, k6 10 VU·30초, 600요청당 상품 목록 SELECT 관측값 591~595회 → 워밍 후 0회 (`ops/perf/2026-10-29-cache-ab/summary.json`). 100원 테스트 결제와 구매 분석 DEMO 시드는 자동 연결되지 않습니다.

개발자 소개

백엔드 중심의 풀스택 개발을 기본 포지션으로 둡니다. 화면, API, 데이터베이스, 캐시, 배포가 어디서 갈라지는지를 문제 단위로 설명하는 쪽에 무게를 두었습니다.

> 경력과 학력은 운영 중인 포트폴리오 서비스의 공개 이력 API에 등록된 내용입니다. 별도 재직 증명으로 대조하지는 않았습니다.

등록된 경력

조이튠 · 웹 개발 · 2022.02 – 2023.12

등록 내용: Java, Spring, JavaScript, MySQL, React, MariaDB, Linux. 제약사 맞춤 웹의 로그인·회원·관리자 화면, 스마트 우편함 관리 화면과 서버 점검 자동화.

학력

경기대학교 수학과 학사 · 2016.03 – 2021.08

이 문서에서 보여주는 기술

Java 21 | Spring Boot 4.1 | Spring Security | JWT | JPA | MySQL | Redis | Next.js 16 | React 19 | TypeScript | Zustand | Docker | Nginx | AWS EC2 · RDS · S3 | GitHub Actions

기여를 적는 기준

- 공개 저장소의 커밋 작성자는 Git 기록상 KimSunWooo입니다. 줄 단위로 직접 작성 비율을 저장소만으로 확정하지 않습니다.
- 웹 포트폴리오 서비스에서 AI 코딩 도구를 어느 범위까지 썼는지는 기록으로 확정되지 않아 단정하지 않습니다.
- 자동매매와 AI 투자 판단 에이전트는 지원자가 Cursor AI 바이브코딩으로 개발했다고 밝힌 프로젝트입니다. 운영 사이트 프로젝트 카드에도 같은 취지가 있습니다.

2026년 프로젝트 요약

프로젝트	역할 관점	현재 상태
웹 포트폴리오 서비스 KimSunWooo/portfolio	Spring Boot API, Next.js, 인증, 장바구니, 캐시, EC2 배포	공개 서비스 운영 중. 장바구니 DB 멱등·인증 경쟁·배포 복구는 코드와 테스트로 검증
KIS 자동매매 KimSunWooo/Auto-Trading-system	Next.js 웹앱, 주문 상태, 멱등성, Worker	Cursor 바이브코딩. KIS PAPER에서 개발·검증 중. REAL 주문은 잠금
AI 투자 판단 에이전트 KimSunWooo/Auto_Trading_AI_Agent	주문 실행과 분리된 판단 API를 의도한 독립 저장소	이번 환경에서 저장소 코드를 열지 못함. 구현 완료로 적지 않음
Python 영상·검출	프레임 버퍼, 프로토콜, 파이프라인, HTTP/WebSocket	웹 직무의 보조 사례. 모델 성능은 다루지 않음

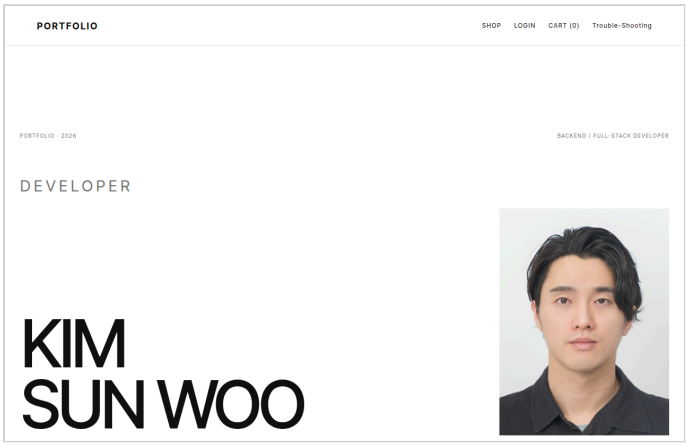
문서에서 구분하는 것

- **코드로 확인한 구현**과 **테스트 코드**, **운영 화면에서 본 동작**을 나눕니다.
- README나 커뮤니티 글의 개선율은 측정 원본이 저장소에 없으면 성과 수치로 쓰지 않습니다.
- 100원 결제와 자동 취소는 테스트 결제입니다. 매출이나 운영 결제 성과가 아닙니다.
- 자동매매의 REAL 매매와 수익률은 다루지 않습니다.

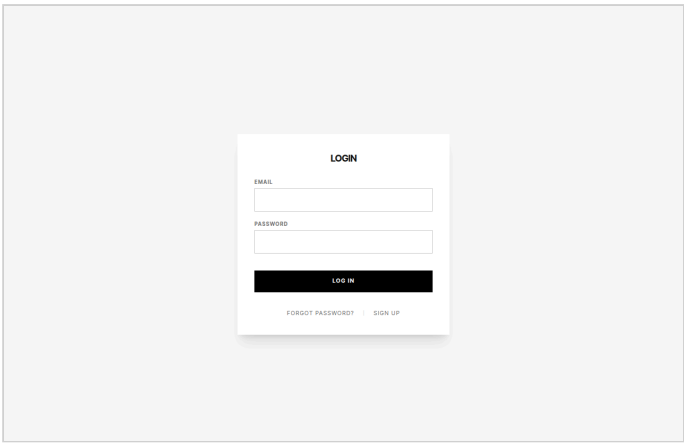
자동매매 클론은 2026-10-12 커밋 52d1aaf 기준입니다. 웹 포트폴리오의 장바구니 DB 멱등은 main의 CartService·CartSyncMySqlTest·merge-retry 브라우저 테스트로 확인합니다.

운영 중인 웹 서비스

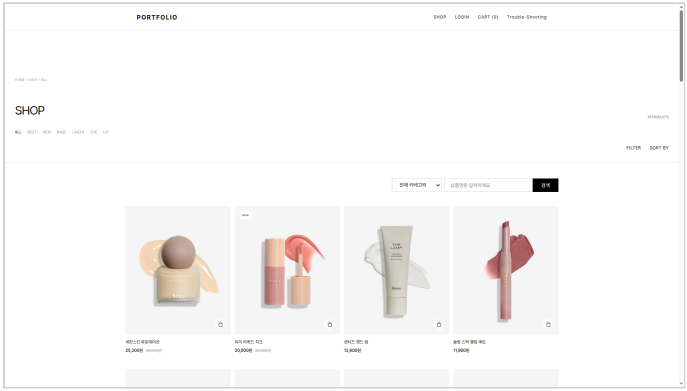
정적 소개 페이지가 아니라 이력·프로젝트·상품·장바구니·커뮤니티를 API로 읽고 관리자 권한으로 고치는 서비스입니다. 상품 데이터는 인증, 소유권, 캐시, 결제 상태 전이를 다루기 위한 모듈입니다.



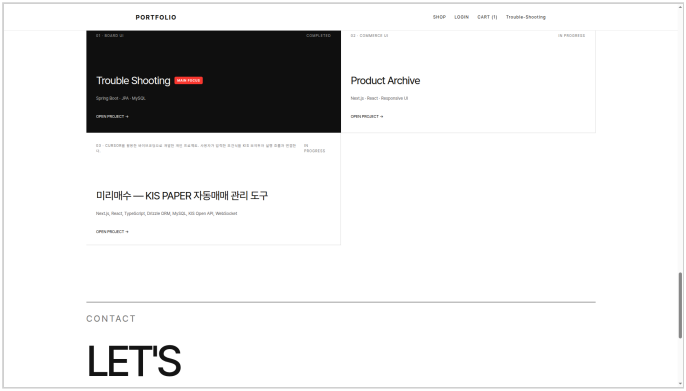
메인. 공개 서비스 swportfolioservices.com, 2026-10-13 캡처.



로그인. Access는 응답 본문, Refresh는 HttpOnly 쿠키로 내려갑니다.

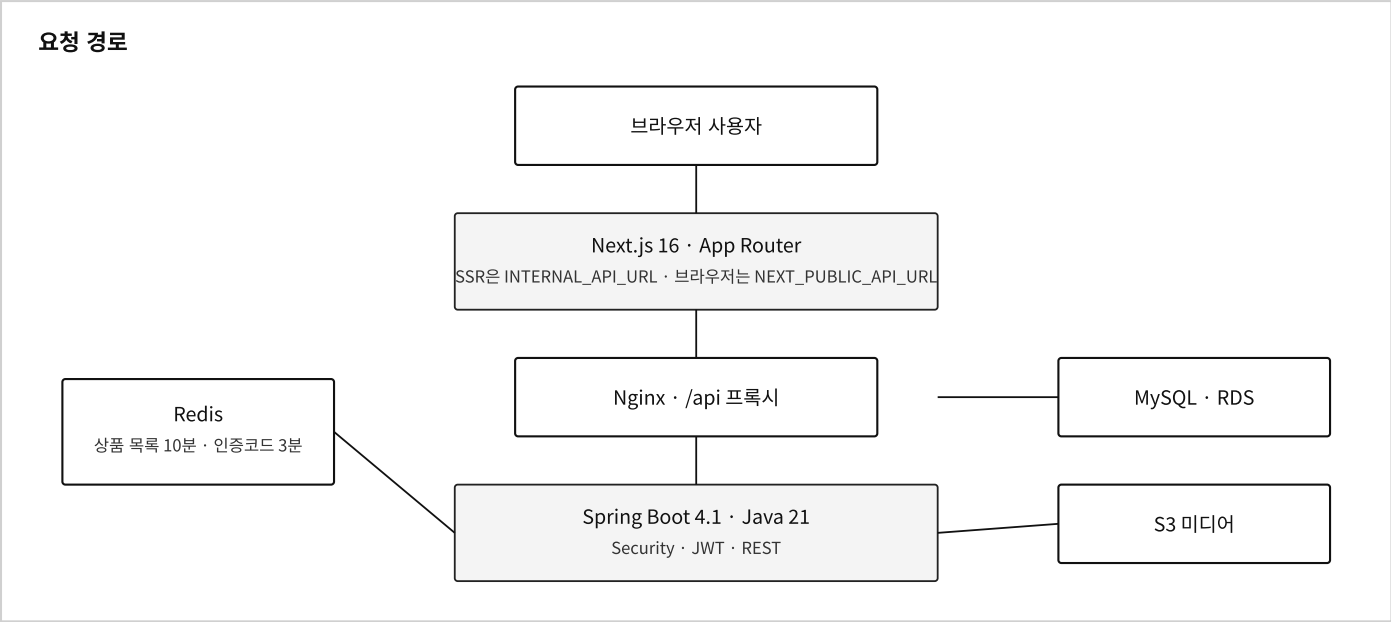


상품 목록. 공개 GET /api/products. 캡처 시점 38개.



홈의 프로젝트 카드. 자동매매는 바이브코딩·PAPER·진행 중으로 표시됩니다.

서비스 구성



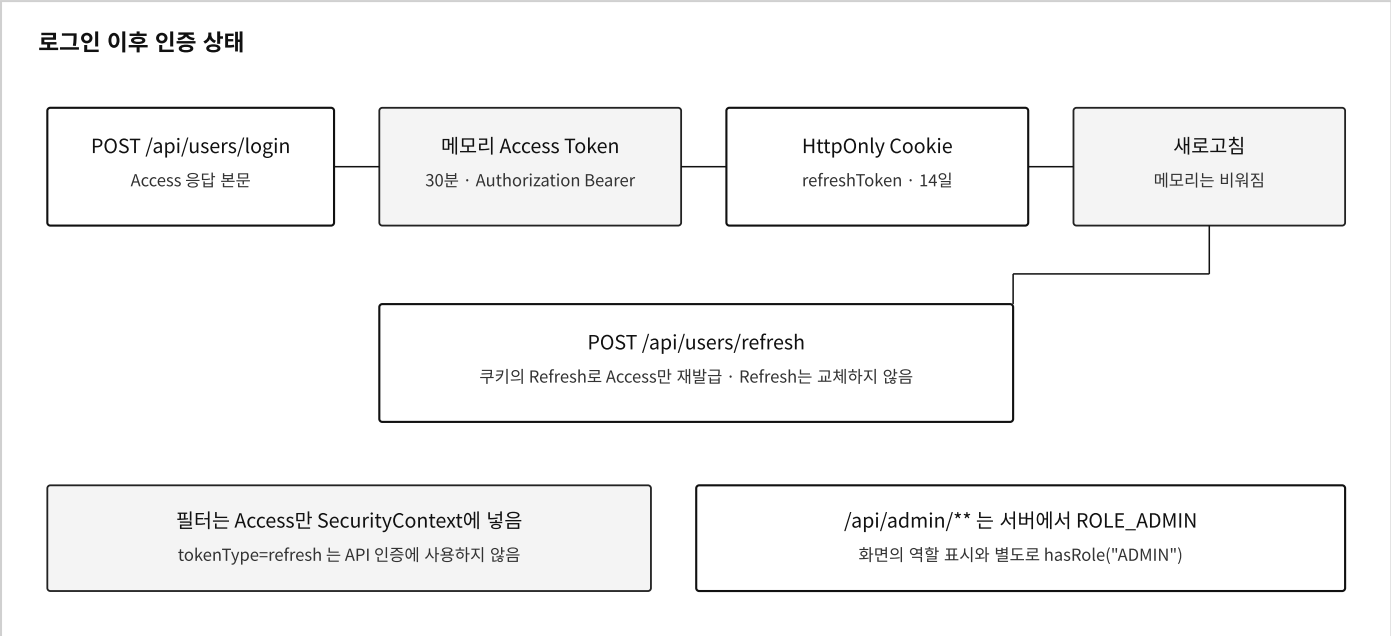
브라우저, Next.js, Nginx, Spring Boot, Redis, MySQL, S3. 코드와 배포 스크립트를 기준으로 그린 구성입니다.

요청이 갈라지는 지점

- 브라우저의 API 주소는 `NEXT_PUBLIC_API_URL`, 서버 렌더의 주소는 `INTERNAL_API_URL`입니다. `frontend/src/lib/api.ts`
- 배포 시 Nginx `/api`는 `service-url.inc`의 upstream으로 프록시합니다. `deploy.sh`, `ops/nginx/rewrite_api_proxy.py`
- 백엔드와 프론트는 서로 다른 컨테이너로 올리고, 슬롯을 바꿔 Nginx를 reload하는 경로가 `deploy.sh`에 있습니다. 진행 중 배포는 취소하지 않도록 workflow concurrency가 걸려 있습니다.
- GitHub Actions는 배포 스크립트 검사, Gradle 테스트, 프론트 lint·타입·테스트·빌드 뒤에 이미지를 푸시하고 EC2에서 해당 커밋의 `deploy.sh`를 실행합니다.

이번 문서 작업에서 GitHub Actions 배포를 다시 실행하지는 않았습니다. 위 내용은 workflow와 스크립트에 있는 절차입니다.

인증과 데이터 소유권



Access와 Refresh를 나눈 이유: 짧은 호출 권한과 갱신 수단을 분리하고, 갱신 토큰은 자바스크립트가 읽지 못하게 합니다.

문제

새로고침하면 메모리의 Access Token이 사라집니다. 관리자 화면만 역할을 믿으면 API는 통과할 수 있습니다.

구현

- Access 30분, claim `tokenType=access`, role 포함.
  `JwtTokenProvider`
- Refresh 14일, HttpOnly·Secure·SameSite=Lax.
  `UserController`
- 필터는 Access만 인증 객체로 넣습니다. Refresh로 관리자 API를 호출하면 통과하지 않습니다.
- `/api/admin/**` 는 `hasRole("ADMIN")`. 권한 문자열은 `ROLE_ADMIN`.

실패했을 때

쿠키가 없거나 Refresh 검증에 실패하면 401입니다. 프론트는 복구 실패와 비로그인을 구분해 안내합니다.

`ensureAccessToken`

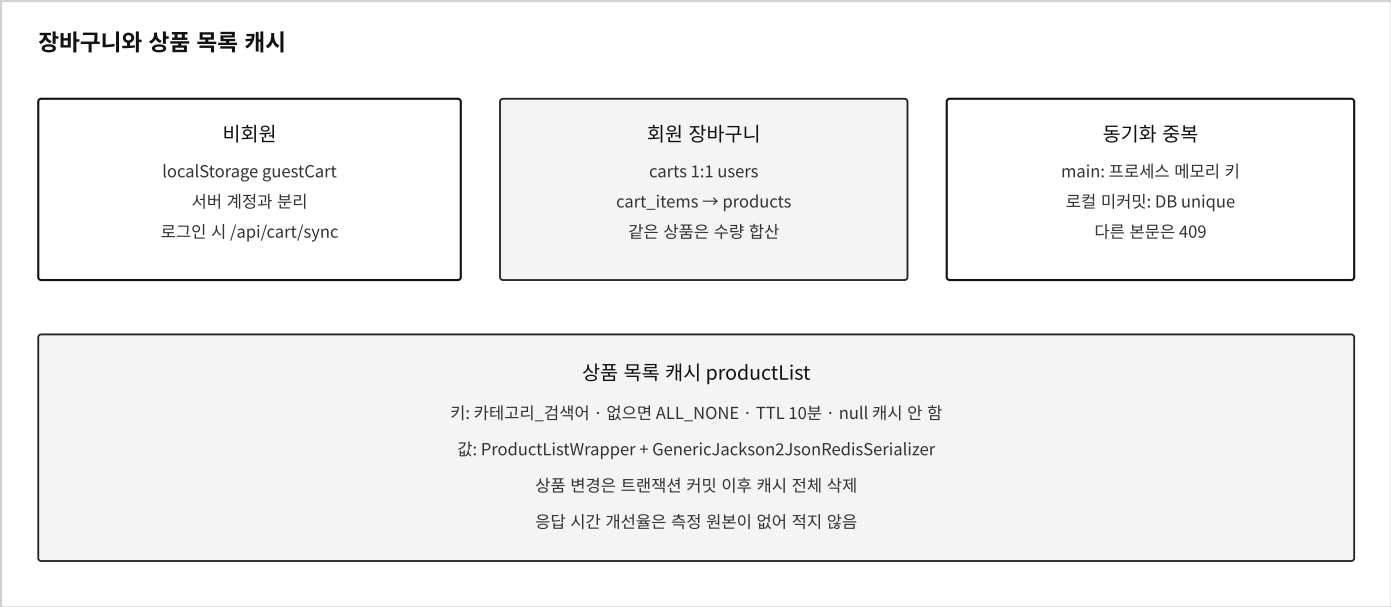
검증

`TokenUseTest`, `AdminAccessWebTest`가 역할과 Refresh 거부를 검사합니다. 이번 세션에서 Gradle을 다시 실행하지는 않았습니다.

데이터

사용자는 이메일 unique. 장바구니는 사용자당 하나. 상품·이미지·상세는 JPA로 나뉘고, 목록 조회는 SALE 상태만 반환합니다. 장바구니 항목 조회는 소유 이메일을 확인합니다.

캐시, 장바구니, 테스트 결제



비회원 장바구니와 회원 장바구니, 그리고 상품 목록 캐시의 경계.

Redis를 쓴 이유와 직렬화

상품 목록은 같은 조건으로 반복 조회됩니다. 캐시 이름은 `productList`, 키는 정규화한 카테고리와 검색어입니다. 제네릭 리스트가 타입 없이 복원되는 문제를 피하려고 응답을 `ProductListWrapper`로 감싸고, Redis 전용 ObjectMapper에 JavaTimeModule과 타입 정보를 넣었습니다. 상품 변경은 커밋 이후에 캐시를 비웁니다. `ProductListCacheTest`가 키와 삭제 시점을 검사합니다.

비밀번호 재설정 코드는 별도 키 `AUTH_CODE:` 로 3분만 저장합니다. 상품 캐시와 용도가 다릅니다.

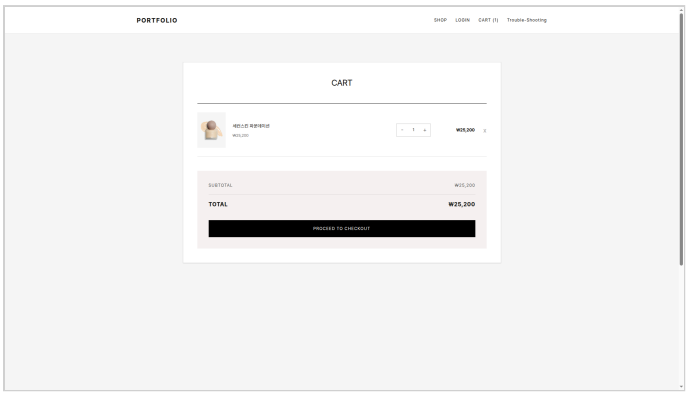
장바구니 동기화 — DB 멱등

`cart_sync_requests` unique (user_email, operation, request_key)와 본문 해시로 재전송을 거절하거나 같은 요청으로 인정합니다. `CartService`, `CartSyncMySqlTest`, Playwright `merge-retry.spec.ts`로 검증합니다.

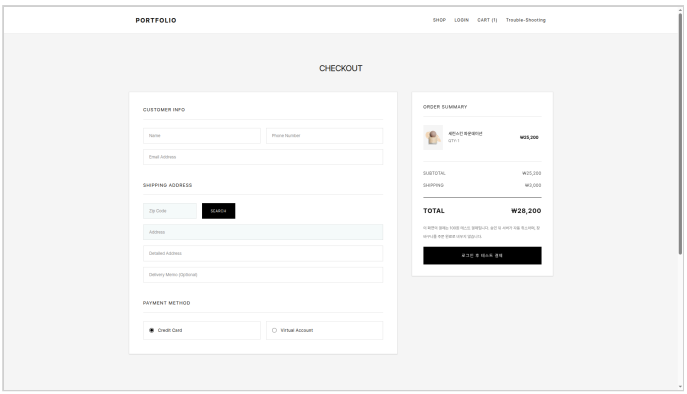
100원 테스트 결제

`PaymentService`는 금액이 100원이 아니면 거절합니다. 승인 응답이 요청과 같아야 DONE을 기록하고, 이어서 자동 취소합니다. 승인 실패 시 DONE을 쓰지 않습니다. 취소가 실패하면 DONE 행은 남기되 완료 응답으로 끝내지 않습니다. 다른 사용자 결제 키는 403입니다. 화면에도 자동 취소와 장바구니 유지가 적혀 있습니다.

화면 연동과 배포에서 고친 문제



비회원 장바구니. 상품을 담은 뒤 2026-10-13 캡처. 수량은 이 브라우저의 localStorage입니다.



체크아웃. 100원 테스트 결제이며 승인 뒤 자동 취소라고 화면에 표시됩니다.

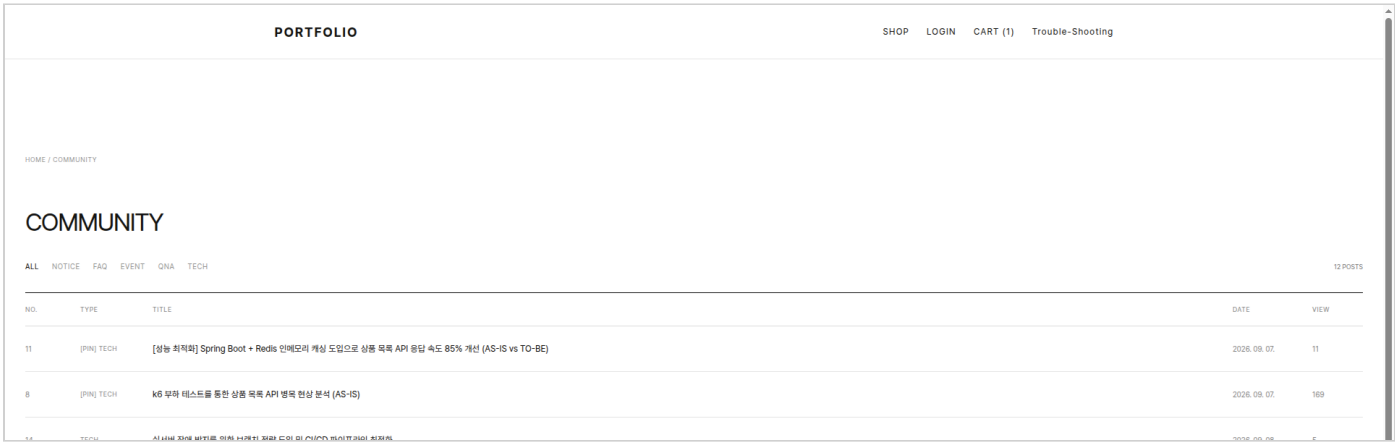
문제 1. 컨테이너 안의 localhost

브라우저의 localhost와 Next.js 컨테이너의 localhost는 다른 곳입니다. SSR이 컨테이너 자신을 8080으로 치면 Connection Refused가 납니다. 렌더 위치에 따라 API 주소를 나누고, 배포 스크립트는 SSR이 백엔드 컨테이너로 붙도록 주소를 넘깁니다. 이미지 URL도 브라우저가 해석할 수 있는 공개 주소로 맞춥니다.

문제 2. 새로고침 이후의 로그인과 업로드

Access Token은 메모리에만 있습니다. 새로고침 뒤에는 Refresh 쿠키로 Access를 다시 받고, 그 토큰으로 보호 API를 호출합니다. 관리자 여부는 서버 규칙이 최종입니다. FormData를 보낼 때는 Content-Type을 지워서 브라우저가 boundary를 붙이게 합니다. `frontend/src/lib/api.ts`

문제 3. 커뮤니티에 적힌 수치



트러블슈팅 글 목록. 제목의 개선율은 글의 주장이고, 측정 원본이 저장소에 없어 이 PDF의 성과 수치로 사용하지 않습니다.

KIS PAPER 자동매매 웹 애플리케이션

개발 방식: Cursor AI를 활용한 바이브코딩 기반 개인 프로젝트
현재 상태: KIS PAPER 환경에서 개발 및 검증 진행 중

투자 전략 소개가 아니라, 로그인·계좌 연결·조건 입력·주문 상태·백그라운드 루프를 가진 Next.js 웹앱으로 봅니다. 패키지명은 `mirae-autobuy`, Next.js 16.3.5, React 19, Drizzle ORM, MySQL입니다. 별도 Express 서버는 없고 `app/api` Route와 `src/` 런타임이 서버 처리입니다.

바이브코딩에서 확인된 범위

운영 사이트의 프로젝트 카드가 이 저장소를 Cursor 바이브코딩 개인 프로젝트로 소개합니다. 저장소 README 자체에는 “바이브코딩”이라는 문장은 없습니다. 줄 단위로 AI가 생성한 코드와 사람이 고친 코드를 가르는 기록은 이번 분석에서 찾지 못했습니다.

코드와 문서에서 확인할 수 있는 작업물은 요구 경계, PAPER/REAL 잠금, 주문 상태 모델, Worker lock, startup sync, 그리고 `npm test`용 테스트 파일입니다. 검증 문서 `docs/VERIFICATION_WORKER_ALLOCATION_2026-10-12.md`는 당시 커밋 8f76b69에서 642 pass / 0 fail / 21 skip을 적고 있습니다. 그 명령을 이번 환경에서 다시 실행하지는 않았습니다.

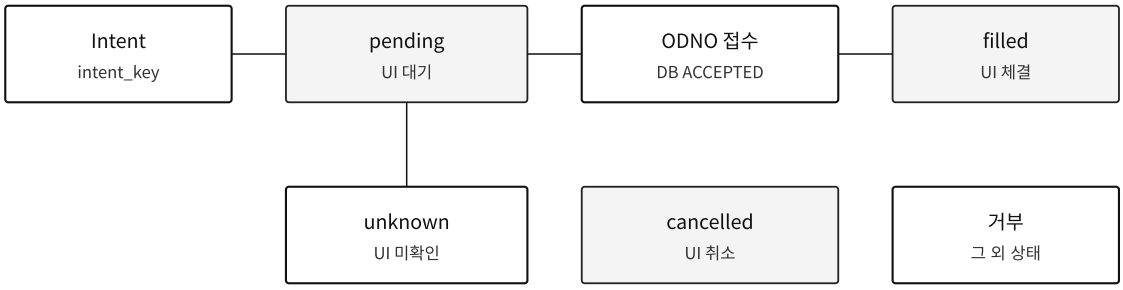
화면과 서버

- 로그인·세션: `app/login`, `app/api/auth/*`, `src/auth/session.ts`
- PAPER 계좌 연결·검증: `app/mypage`, `/api/accounts/paper`
- 대시보드·조건·적립·주문 내역: `components/trading-app.tsx`와 각 패널
- 시세·잔고 갱신: `hooks/use-trading.ts`
- 기동 시 Worker: `instrumentation.ts`에서 PAPER 런타임 루프

이 환경에는 자동매매 서버를 띄우지 않았습니다. KIS 주문 API도 호출하지 않았습니다. 대시보드 스크린샷은 없어 아래 요청 문서에 따로 적습니다.

주문 상태, 멱등성, 재시작

주문 의도와 체결을 분리한 상태



타임아웃·제출 후 DB 기록 실패는 unknown. 접수 번호만으로 체결 완료가 되지 않는다.

같은 계좌의 intent_key, 주문번호, 체결 키에 unique 제약이 있다.

Worker lock을 잡지 않으면 주문을 진행하지 않는다. REAL 주문은 여러 플래그가 모두 있어야 한다.

의도, 접수, 체결을 같은 완료 상태로 합치지 않습니다. UI 배지는 components/orders-panel.tsx 기준입니다.

중복과 불확실

- 주문 의도는 `order_intents.intent_key` 로 계좌 안에 유일합니다.
- 로컬 주문 ID, 증권사 주문번호, 체결 키도 unique입니다.
- HTTP 타임아웃이나 제출 후 DB 실패는 `unknown`입니다. 미확인이 있으면 새 주문을 막는 검사가 테스트에 있습니다.
- HTTP 헤더 이름의 Idempotency-Key는 없습니다. 런타임의 키는 intent id입니다.

프로세스와 복구

- Worker lock 파일 기본 TTL은 8초입니다. lock이 없으면 주문 사전 검사가 막습니다. `src/runtime/worker-lock.ts`
- DB가 장부의 기준이고, 기동 시 KIS의 열린 주문·체결과 로컬 상태를 분류합니다. `startup-sync.ts`
- REAL 주문은 모드, 허용 플래그, 확인 문구가 맞을 때만 열리도록 테스트가 잠금을 검사합니다. 기본 예시는 PAPER이고 REAL은 README에서 LOCKED입니다.

화면에 완료라고 쓰지 않는 부분

`pending`은 대기, `unknown`은 미확인, 체결 수량이 주문 수량보다 적으면 부분체결입니다. 주문번호만 받은 상태를 DB에서는 ACCEPTED로 두고 FILLED와 구분합니다. 탭 이름과 빈 목록 문구는 “체결”에 가깝게 적혀 있어, 대기 주문이 있어도 문구만 보면 체결 내역처럼 읽힐 여지는 있습니다. 배지 값은 미체결과 완료를 나눕니다.

테스트와 아직 운영이 아닌 범위

구분	근거	이번 문서에서의 의미
단위·모의	FakeBroker, mock fetch, `tsx --test` 파일 다수	상태 전이, lock, unique, REAL 잠금을 코드로 고정
검증 문서의 일괄 실행	2026-10-12 문서, 커밋 8f76b69, 642 pass	그 시점의 기록. HEAD 52d1aaf에서 재실행하지 않음
PAPER API	README와 opt-in 스크립트. 예: 국내 VTS 주문번호 서술	클론 안에 실행 로그는 없음. 실전 매매 증거가 아님
REAL	코드 경로는 있고 기본값은 잠금	운영한 것으로 적지 않음
장시간 실행	README의 soak·WebSocket 항목은 조건부이거나 미완료 표현	완료된 무중단 운영으로 적지 않음

면접에서 말할 수 있는 설계

- 주문을 내기 전에 의도와 로컬 행을 남기고, 증권사 응답이 애매하면 성공으로 덮지 않습니다.
- 같은 의도가 두 번 주문이 되지 않도록 DB unique와 열린 매수 차단이 같이 있습니다.
- 프로세스가 죽으면 lock TTL이 지나고, 다음 기동은 DB와 증권사 조회로 상태를 다시 분류합니다.
- AI 제안은 이 저장소에서 주문 자격이 기본 거부입니다. `autoOrderEnabled` 기본값은 false이고, 활성화 API는 403으로 막혀 있습니다.

긴급 정지 UI는 cancel과 flatten을 함께 호출합니다. README가 말하는 stop과 flatten의 분리와 화면 동작이 완전히 같지는 않습니다.

AI 투자 판단 에이전트

> **개발 방식: Cursor AI를 활용한 바이브코딩 기반 독립형 프로젝트**
> 지원자가 명시한 개발 방식입니다. 저장소 코드를 이번 환경에서 열지 못해, 아래는 생산 측 구현 완료 서술이 아닙니다.

확인하지 못한 것

`https://github.com/KimSunWooo/Auto_Trading_AI_Agent` 는 인증 없는 GitHub API에서 404였습니다. 비공개이거나 이름이 다를 수 있습니다. 이 머신에는 GitHub CLI와 credential helper가 없었습니다. 소스, 테스트, Fail-closed 구현, TradeProposal 생성 코드는 보지 못했습니다. 자동매매와 붙어서 운영 중이라고 적지 않습니다. 실계좌 주문도 적지 않습니다.

공개된 소비 측 계약

자동매매 저장소 `docs/AI_CONTRACT.md` 는 별도 프로세스가 제안을 만들고, 자동매매가 받아서 주문 여부를 결정하는 경계를 적습니다. 스키마 이름은 `ai.trade-proposal.v1` 입니다.

- 제안의 dataMode는 FIXTURE, REPLAY, LIVE_OBSERVATION, LIVE_SHADOW뿐이며, 이 모드들은 주문 자격이 없다고 적혀 있습니다.
- 생산 측이 주문을 표시해도 소비 측이 거절합니다. HOLD 계열 행동은 매수·매도로 바꾸지 않습니다.
- 선호 전송은 REST pull입니다. Push 경로는 없다고 적혀 있습니다.
- 자동 주문 활성화는 기본 거부입니다.

이 계약은 자동매매 저장소에 있는 소비 측 문서입니다. 에이전트 저장소가 같은 계약을 구현했는지는 미검증입니다. 비공개 저장소를 포트폴리오에 넣으려면 공개 범위를 정한 뒤 다시 분석해야 합니다.

Python 프로젝트에서 가져온 기술 사례

모델 구조와 정확도는 적지 않습니다. 웹 개발과 닿는 동시성, 통신 계약, 파이프라인만 한 장으로 둡니다. 서로 다른 저장소입니다.

사례	코드에서 본 것	상태
안전모 동작 프로토타입 Motion-detected-system ec0737c	`LatestFrameBuffer`가 lock 아래 프레임을 최대 1장만 유지하고, 새 프레임이 오면 읽지 않은 프레임을 버립니다. 처리가 느릴 때 큐가 쌓이지 않게 하는 구조입니다.	대시보드는 데모 시나리오입니다. 정식 서비스가 아니라 착수 전 프로토타입으로 분류합니다. 객체 기반 착용 검출기는 더미입니다.
돌봄 UI DolBom · 4d11607	PyQt 클라이언트와 AI를 분리합니다. 제어는 길이 헤더와 JSON, 영상은 JPEG RTP/UDP 초안입니다. 카메라 역할은 `CameraHub`가 나눕니다.	프로토콜 상태는 draft, 기본은 데모입니다. 병원 연동이나 녹화 완료로 보지 않습니다.
검출 메인 서버 deep-learning-project e6565dd	카메라, 메인, AI, UI가 WebSocket과 HTTP로 나뉩니다. 프레임은 JSON 헤더와 JPEG입니다. 카메라 ID가 여러 개입니다.	DolBom은 환경 변수로 이 서버의 미리보기·결과 소켓을 볼 수 있습니다. 선택 연동 코드이며, 통합 운영 완료로 적지 않습니다.
차량 번호 파이프라인 vehicle_detect_system 7e906a2	검출, 분류, 번호판, OCR을 단계로 잇고 실패 시 빈 OCR 결과를 남깁니다. CLI가 `output/results.json`을 씁니다.	이 저장소 안에 HTTP 인식 서버나 DB 적재는 없습니다.

문제 해결을 한곳에 모으면

문제	선택한 처리	확인
SSR과 브라우저의 API 호스트가 다름	렌더 위치별 URL, Nginx upstream, 공개 이미지 주소	api.ts, deploy.sh. 커뮤니티 글은 당시 장애 기록
Refresh로 관리자 API가 열릴 수 있음	tokenType과 role로 Access만 인증, admin 경로는 서버 역할 검사	SecurityConfig, TokenUseTest, AdminAccessWebTest
목록 캐시가 타입을 잃고, 수정 전에 비워질 수 있음	Wrapper, 전용 Jackson, 커밋 후 삭제, TTL 10분	RedisConfig, ProductListCache. 속도 수치는 제외
비회원 장바구니 재전송이 수량을 중복 합산	main은 프로세스 내 키. 로컬 작업은 DB unique와 본문 해시	CartSyncMySqlTest·merge-retry로 검증. 운영 배포는 main 경로
테스트 결제를 성공 주문으로 오해	100원만 허용, 승인 실패 시 DONE 없음, 취소 실패는 완료 응답 아님	PaymentService, 체크아웃 문구, PaymentServiceTest
주문 타임아웃을 체결로 기록	unknown, unique intent, worker lock, 기동 시 재분류	자동매매 소스와 테스트. REAL 운영 증거 없음
영상 프레임이 처리보다 빨리 쌓임	최신 1장만 유지하고 나머지를 drop	안전모 프로토타입의 LatestFrameBuffer

공통으로 둔 기준은 경계에서 상태를 나누고, 애매한 결과를 성공으로 승격하지 않는 것입니다. 화면 문구와 서버 상태가 어긋나면 서버 상태와 테스트를 우선합니다.

링크와 읽는 순서

이름	주소	볼 내용
웹 서비스	swportfolioservices.com	이력, 상품, 장바구니, 커뮤니티
포트폴리오 저장소	github.com/KimSunWooo/portfolio	Spring Boot, Next.js, 배포
자동매매	github.com/KimSunWooo/Auto-Trading-system	PAPER 웹앱, 주문 정합성. 바이브코딩
안전모 프로토타입	github.com/KimSunWooo/Motion-detected-system	최신 프레임 버퍼
돌봄 UI	github.com/KimSunWooo/DolBom	UI와 통신 초안
검출 서버	github.com/KimSunWooo/deep-learning-project	HTTP·WebSocket 경계
차량 파이프라인	github.com/KimSunWooo/vehicle_detect_system	단계별 JSON 결과

같이 보면 좋은 파일

- `SecurityConfig`, `JwtTokenProvider`, `UserController`
- `ProductListCache`, `RedisConfig`, `CartService`, `PaymentService`
- `frontend/src/lib/api.ts`, `.github/workflows/deploy.yml`, `deploy.sh`
- 자동매매 `components/orders-panel.tsx`, `src/runtime/worker-lock.ts`, `src/runtime/startup-sync.ts`, `docs/AI_CONTRACT.md`

김선우 · Backend / Full-stack · 2026

관리자 화면과 자동매매 대시보드, AI 에이전트 저장소는 이번 PDF에 포함하지 않았습니다. 필요한 추가 캡처는 SCREENSHOT_REQUESTS.md에 있습니다.